

READY-TO-USE PROMPT / 01

OpenBEXI Timeline rebuild brief

Reverse engineer the complete application. Preserve its capabilities, improve its usability and performance, and deliver a thoroughly verified implementation.

Role and mission

Act as a senior reverse-engineering specialist, full-stack architect, interactive timeline developer, UX designer, and software quality engineer. Analyze the repository below, then implement an improved version of OpenBEXI Timeline:

https://github.com/arcazj/openbexi_timeline

Preserve the application's verified functionality and data compatibility while making it faster, more reliable, easier to maintain, and more intuitive. Provide both an interactive **Gantt timeline** and an equivalent **tabular view** of the same data. Write all authored project documentation in **Markdown**, with a simple, practical README.md.

1. Establish the complete baseline before redesigning

- Obtain the repository and record the branch and exact commit SHA. Inventory all tracked files. Read every first-party source file and relevant configuration, test, example, and documentation file. Classify generated, vendored, and binary files; explain exclusions. Record inaccessible or unreviewed material explicitly.
- Inspect frontend and backend code, rendering and interaction logic, data models, persistence, APIs, integrations, dependencies, build scripts, and deployment. Begin with README.md, package.json, pom.xml, the HTML entry points, Dockerfile, launch scripts, and the source, tests, data, configuration, and documentation directories actually present.
- Run the original application in a reproducible local environment when possible. Exercise its screens and workflows, inspect representative datasets, and capture baseline behavior, screenshots, errors, and performance. If execution is blocked, document the reason and distinguish source-derived conclusions from observed behavior.
- Create a feature inventory and traceability matrix linking each capability to source locations, current behavior, replacement behavior, and verification. Separate intentional behavior, defects, incomplete features, and proposed enhancements. Do not infer an active feature merely from a dependency declaration.

Working rule: complete discovery before committing to the architecture. Continue through implementation, validation, and documentation; keep findings and progress in Markdown. Resolve routine choices autonomously and document assumptions.

Architecture and compatibility

Use the original application as the functional reference and the evidence from the audit to choose the implementation.

2. Design a maintainable replacement

- Explain the current architecture, data flow, module responsibilities, and main weaknesses. Compare a focused refactor with a broader rebuild. Select the simplest approach that meets the requirements and document the decision and tradeoffs.
- Evaluate the existing JavaScript rendering stack and Java build/backend components before replacing them. Verify actual runtime use. Choose maintained dependencies and reproducible versions; justify any framework, renderer, database, or protocol change with compatibility, performance, and maintenance evidence.
- Separate domain data and time calculations from rendering, view state, data access, and persistence. Make timeline and table components consume shared operations and selectors. Avoid duplicate business rules, uncontrolled global state, circular dependencies, and unnecessary services.
- Keep the original implementation available for comparison. Work in an isolated branch or project directory and deliver runnable increments: baseline and audit, shared model, timeline, table, integration, optimization, tests, and documentation.

3. Preserve data and integration contracts

- Document actual event identifiers, date formats, durations, groups, metadata, configuration, filters, API contracts, persistence rules, and live-update mechanisms. Preserve all verified capabilities and supported inputs, or supply a documented, tested compatibility adapter or migration.
- Use stable record identifiers and one authoritative domain model. Define validation, required and optional fields, null values, date precision, interval boundaries, timezone conversion, and duration calculations. Preserve meaningful legacy semantics and unknown metadata during round trips.
- Make timezone display explicit. Handle UTC, local zones, daylight-saving transitions, zero-duration events, overlapping intervals, open-ended intervals when supported, malformed dates, and very long time ranges. Never silently reinterpret ambiguous timestamps.
- Distinguish read-only and writable data sources. Preserve supported create, read, update, and delete operations and their permissions. Validate edits consistently, persist through the proper source, surface failures, and prevent silent data loss or stale responses overwriting newer state.

Scope discipline: implement the existing product and the requested improvements. Verify whether scheduling dependencies, progress percentages, recurrence, or other project-management concepts actually exist before including them. Do not substitute an unrelated generic Gantt application.

READY-TO-USE PROMPT / 03

A clearer, more interactive UI

Make common actions easy to discover and keep the timeline readable, responsive, and useful with real data.

4. Improve the complete user experience

- Create a coherent interface with clear navigation, readable typography, consistent spacing, visible selection, accessible colors, and helpful labels. Prioritize everyday tasks: loading data, finding an event, navigating time, inspecting details, filtering, and switching views.
- Provide intuitive zoom and pan, sensible time-scale changes, a fit-to-data action, navigation to a selected event, and a visible current-time indicator where meaningful. Start with a useful viewport and keep navigation stable when resizing or changing views.
- Preserve verified grouping, lanes or bands, event styling, legends, configuration, multiple timelines, and other discovered controls. Improve dense and overlapping event layouts, label readability, and selection hit areas without hiding information silently.
- Provide useful event details through a consistent inspector, panel, or dialog. Implement drag, resize, inline editing, and other mutations only where the domain and data source support them. Offer precise form or keyboard alternatives to pointer gestures.
- Make loading, empty, invalid-data, disconnected, and error states explicit and actionable. Preserve useful content during refreshes. Avoid unexplained blank screens, unstable layouts, and controls that appear to work but have no implemented behavior.

5. Support the devices and interaction methods people use

- Adapt the interface to desktop, tablet, and smaller screens. On narrow screens, use a usable single-view layout and deliberate horizontal scrolling where needed. Ensure touch gestures do not conflict with ordinary page navigation.
- Support keyboard navigation, clear focus indicators, accessible control names, logical focus management, and screen-reader access to event details and data. Target WCAG 2.2 AA for applicable UI criteria and report the checks performed.
- Use color together with text or symbols to communicate status. Keep contrast and text legibility sufficient. Respect reduced-motion preferences and provide accessible alternatives for any canvas or WebGL interaction.
- Preserve sensible user preferences, including display timezone, chosen view, and table columns. Explain which settings survive refresh or are shared in links. Keep implementation details out of ordinary product flows.

UI acceptance: demonstrate the main user journeys with screenshots and browser checks. Verify realistic populated screens as well as empty, loading, and failure states; a polished screenshot alone is not evidence of a working interface.

READY-TO-USE PROMPT / 04

One dataset, two synchronized views

The tabular view is a full working representation of the timeline data, with consistent meaning and behavior.

6. Build the table and synchronization contract

Provide **Timeline** and **Table** modes with an obvious view switch. Add a resizable side-by-side mode when screen space permits. Both views must read the same canonical records and use the same validation and data operations.

Concern	Required behavior
Record identity	A table row and its timeline event refer to the same stable ID. Show matching totals for the same filter and scope.
Search and filters	Apply shared search, data-source, group, and date filters. Distinguish the viewport from the data filter; make any visible-range-only table mode explicit.
Selection and navigation	Selecting an event selects its row; selecting a row identifies its event. Provide a reliable reveal-in-timeline action without unexpected viewport jumps.
Edits and deletion	Where supported, either view invokes the same operation. Update both views without a page reload, show save status, and reconcile or roll back failed changes.
Live data and refresh	Apply incoming changes once to the shared model. Handle duplicates, stale responses, reconnects, and deleted records consistently.
Switching and sorting	Preserve filters, selection, and useful navigation state. Table sorting must not unintentionally reorder timeline lanes or reset the time range.

7. Make the table useful for inspection and work

- Derive columns from the verified schema. Include the record identifier, title or label, start, end, and duration when meaningful; expose actual groups, status, source, and metadata when available. Clearly mark missing or non-applicable values.
- Provide typed sorting, search, filtering, configurable columns, column resizing, readable date formatting, and record details. Use pagination or row virtualization when measurements justify it. Define stable ordering and selection behavior across pages or unloaded rows.
- Expose supported editing actions with the same permissions, validation, and error messages as the timeline. Export supported formats consistently; if CSV export is added, document its scope and handle delimiters, Unicode, dates, and spreadsheet formula injection safely.

Synchronization acceptance: test the same actions from both views and verify record IDs, values, counts, persistence, and error recovery. Visual similarity alone does not establish equivalence.

Performance and resilience

Make optimization measurable and preserve correct behavior under load, failures, and repeated use.

8. Establish and meet performance budgets

- Measure the original application before optimizing. Define small, typical, and stress datasets based on expected use, including dense time windows, long labels, many groups, and sustained updates when supported. Use reproducible fixtures and fixed random seeds.
- Record hardware, operating system, browser and runtime versions, build mode, viewport, network conditions, dataset sizes, cache state, and benchmark procedure. Compare equivalent workflows and data in the original and rebuilt applications.
- Measure startup and data-ready time, rendering and update latency, pan and zoom responsiveness, search and filtering latency, table scrolling, memory use, and API or persistence performance where applicable. Use repeated runs and report distributions or median and tail latency where meaningful.
- Set numerical acceptance budgets after establishing the baseline and before evaluating the final implementation. State each target, measurement method, and rationale. Report before-and-after results and tradeoffs; never describe performance gains that were not measured.
- Profile first, then address demonstrated bottlenecks. Consider viewport culling, table virtualization, incremental rendering, indexing, batching, caching, request cancellation, and background computation as needed. Avoid redundant full renders and unbounded caches.
- Check repeated loading, filtering, switching, resizing, and reconnecting for resource leaks. Clean up event listeners, timers, workers, subscriptions, and renderer or GPU resources where used. Keep the interface responsive and explain limits when data exceeds the supported capacity.

9. Handle failure and security at real boundaries

- Validate external data and API input. Safely render untrusted labels, descriptions, URLs, and imported content. Protect supported write operations and preserve existing authentication and authorization boundaries. Keep credentials out of source code, client bundles, and logs.
- Handle network timeouts, malformed responses, partial failures, cancellation, source unavailability, and reconnection. If the application streams updates, define ordering, deduplication, retry, and resynchronization behavior for the actual protocol.
- Document dependency and configuration risks, correct confirmed issues, and use suitable automated checks. Provide useful diagnostics without exposing sensitive data. Make build, startup, configuration, shutdown, and recovery behavior reproducible.

Evidence rule: if the original cannot run, state that a direct comparison is unavailable. Benchmark the replacement against declared budgets and clearly label the limitation; do not invent a baseline.

Testing from end to end

Prove the important behaviors and failure paths with reproducible tests, realistic data, and honest results.

10. Build a layered verification strategy

- **Unit tests:** verify date parsing and conversion, interval logic, sorting, filtering, data normalization, validation, state transitions, and view selectors. Cover invalid inputs, missing fields, duplicate IDs, boundary dates, leap years, daylight-saving transitions, and zero-duration or overlapping events.
- **Integration and contract tests:** verify data loading, adapters, persistence, API formats, configuration, and live updates where present. Test round trips with existing datasets and representative unknown metadata. Exercise timeout, failure, retry, conflict, and reconnection behavior.
- **End-to-end browser tests:** cover clean startup, data loading, search, filters, zoom, pan, details, view switching, synchronized selection, supported edits and deletion, save and reload, and export where available. Verify outcomes against canonical data rather than screenshots alone.
- **Regression and visual tests:** map preserved features and fixed defects to repeatable checks. Test long labels, crowded intervals, many groups, extreme zoom levels, resizing, and representative screen widths. Control fonts, clocks, and test data for stable visual comparisons.
- **Accessibility and compatibility checks:** combine automated checks with keyboard, focus, touch, and screen-reader review where available. Exercise Chrome, Edge, Firefox, and Safari or the available browser engines; record exactly what ran and any actual-browser or device gaps.
- **Performance and robustness tests:** automate the agreed benchmarks and test repeated interactions, large datasets, streaming bursts where relevant, and resource cleanup. Test actual input and permission boundaries rather than relying only on dependency scans.

11. Make every quality claim auditable

- Create a requirements-to-tests matrix covering every preserved feature, requested enhancement, critical workflow, and important failure path. Use meaningful assertions; do not add superficial tests merely to increase a coverage percentage.
- Run formatting, linting, applicable type or static checks, unit tests, integration tests, end-to-end tests, and the production build through documented commands and CI. Verify a clean installation and launch using the README instructions.
- Fix discovered defects and rerun affected checks. Record exact commands, environment, test counts, results, logs or report locations, benchmark results, and manual checks. Distinguish passed, failed, skipped, blocked, and not-run items.

Completion standard: do not claim exhaustive correctness, zero defects, or successful tests without evidence. Critical workflows must pass. Unresolved failures and untested areas must remain visible, with their impact and reproduction steps.

Documentation and final delivery

Deliver an application another developer can install, understand, verify, and maintain.

12. Write practical documentation in Markdown

Keep README.md short and suitable for a newcomer. Include the purpose, prerequisites and versions, installation and startup commands, local URL, sample data, basic timeline and table use, test commands, and links to detailed guides. Verify the instructions from a clean checkout.

Provide these Markdown documents, combining files only when it improves clarity:

Document	Required content
docs/analysis.md	Audit scope, commit SHA, source inventory, existing behavior, defects, assumptions, and exclusions.
docs/architecture.md	Modules, data flow, shared state, rendering, and justified design decisions.
docs/data-model.md docs/api.md	Schemas, time semantics, validation, compatibility, and actual integration contracts; mark API documentation non-applicable if appropriate.
docs/user-guide.md docs/configuration.md	Timeline and table workflows, keyboard controls, settings, data sources, and worked examples.
docs/development.md docs/testing.md	Project structure, build workflow, fixtures, test commands, results, and known verification gaps.
docs/performance.md docs/traceability.md	Reproducible benchmarks and comparisons; requirements, implementation, and verification mapping.
docs/deployment.md docs/migration.md docs/troubleshooting.md	Installation, deployment, rollback, compatibility or migration, common errors, and known limitations.

Maintain CHANGELOG.md and contribution guidance. Preserve license and attribution notices. Link native schemas, API specifications, and generated test reports from Markdown. Use diagrams and screenshots where helpful.

13. Deliver and assess the finished work

Deliver runnable source, build and configuration files, sample data, tests and results, benchmarks, migration support where required, and Markdown documentation. Summarize preserved features, improvements, measured gains, and remaining limitations with evidence links.

Accept completion only when clean setup works, the feature matrix is satisfied, both views stay consistent, supported data survives load/edit/save, critical tests pass, performance meets declared budgets or misses are disclosed, and documentation matches the implementation. Leave no placeholders in required workflows. If blocked, deliver completed work and explain what remains and why.

END OF PROMPT

Preparation note: this is an execution prompt, not a completed audit. Repository structure and manifests informed its scope; the executing agent must inspect the current source.

Source references: [OpenBEXI Timeline repository](#) · [package.json](#) · [pom.xml](#). Prepared 6 September 2026.